04 · 工具调用与执行可靠性

技术面试准备材料 · AI 应用 / Agent 开发

> 核心问题：模型只负责「提议」，系统必须负责「安全、正确、可恢复地执行」。

1. 工具（Tool Schema）应该怎么设计？

我会把工具当成对外 API 来设计，而不是给模型看的说明书。

第一，职责单一。一个工具只做一件事，名字用动词开头，描述说清楚「做什么、什么时候用、什么时候不要用」。

第二，参数少而明确。必填参数尽量少，类型用枚举或 Schema 约束，避免自由文本里塞结构化信息；有副作用的参数要显式标注。

第三，输出结构化。返回统一的成功/失败结构，包含错误码、可读信息、是否需要重试，避免模型从一大段文本里猜。

第四，标注风险等级和成本。只读、可逆写、不可逆写、外部副作用要区分，系统据此决定是否审批、是否沙箱、是否允许重试。

第五，提供示例，特别是边界情况和常见错误。

工具描述的质量直接决定模型选得对不对。很多「模型不会用工具」，其实是工具设计得太模糊。

2. 工具调用失败有哪些类型？分别怎么处理？

我会先分类，再决定是否重试。

第一类是参数错误：缺字段、类型不对、枚举越界。这类通常不自动重试，而是把校验错误返回给模型修复一次，仍失败就转人工。

第二类是瞬时故障：连接失败、读超时、429、5xx。可以重试，但要指数退避加抖动，并受全局预算限制。

第三类是业务错误：余额不足、权限不够、资源不存在。重试没有意义，应该把明确原因返回给模型或用户。

第四类是结果不确定：请求发出去了，但没收到回执。这类不能简单重试，要先查执行记录或幂等状态。

第五类是工具本身不可用：需要熔断、切备用工具或降级。

关键原则是：只重试「可能成功且重试安全」的错误；有副作用的调用必须有幂等键；重试不能层层叠加。

3. 如何设计工具调用的幂等？

核心是给每次「业务上只应发生一次」的操作一个幂等键，并记录它的状态。

幂等键通常由业务标识生成，比如任务 ID + 步骤 ID + 工具名，而不是随机 UUID；同一个逻辑操作重试时必须用同一个键。

状态机一般是：不存在 → PROCESSING → SUCCESS / FAILED。收到请求时先原子地创建或读取状态：如果是 SUCCESS，直接返回缓存结果；如果是 PROCESSING，说明有并发或上次没完成，返回「处理中」或等待；如果是 FAILED，根据错误类型决定是否允许重试。

实现上可以用 Redis + Lua 保证「检查并占位」的原子性，再把最终结果持久化到数据库。注意幂等记录的 TTL 要覆盖业务重试窗口，不能太短。

幂等只保证「重复调用不会产生重复副作用」，不保证「第一次一定成功」。所以还要配合超时、租约和人工兜底。

4. 有副作用的工具执行到一半崩溃，怎么保证不重复执行?

这是 Agent 可靠性里最容易被忽略的问题。

我会把工具调用显式建模成几个状态：PENDING、IN_FLIGHT、SUCCEEDED、FAILED、UNCERTAIN。调用发出后进入 IN_FLIGHT；超过租约还没有回执，就标记为 UNCERTAIN，而不是当成失败。

恢复时不能只看检查点里有没有快照，还要核对工具执行记录：如果记录显示已经成功，就跳过并回填结果；如果显示失败且可重试，就重试；如果是 UNCERTAIN，就根据工具是否幂等决定自动重放、查询下游状态，还是转人工确认。

同时，关键操作本身要尽量幂等，比如带业务唯一键、使用下游的幂等接口。这样即使不确定，重放也是安全的。

这件事的本质是把分布式系统里的「至少一次 / 至多一次」问题搬到 Agent 里。不能假设进程不会崩，也不能假设工具调用是原子的。

5. 如何避免重试风暴?

重试风暴通常来自两层或多层同时重试。比如 SDK 重试一次、网关重试一次、业务层又重试一次，实际请求量会相乘。

我的做法是：只在一个人负责重试。通常把网络层重试收敛到网关，业务层只负责判断「这个错误是否值得重试」，不自己再发一遍。上下层之间通过错误语义约定，避免重复。

其次要有全局预算：每个请求、每个租户、每个模型都有最大重试次数和 Token/成本上限，超过就快速失败或降级。

第三是退避加抖动，避免大量请求在同一时刻重试；对 429 要尊重 Retry-After。

第四是熔断：某类错误比例超过阈值就暂时停止调用，给上游恢复时间。

第五是可观测：记录重试次数、重试成功率、放大倍数，出现异常能及时发现。

重试不是越多越可靠，错误的倍增比失败更危险。

6. 工具执行沙箱需要限制哪些维度?

至少六个维度。

第一，文件系统：只挂载允许的目录，默认只读，禁止访问宿主敏感路径，临时目录用完即删。

第二，网络：默认禁网，需要时走白名单代理，禁止访问内网和元数据服务。

第三，资源：限制 CPU、内存、磁盘、进程数和执行时间，防止死循环、内存爆炸和 Fork Bomb。

第四，权限：以非 root 用户运行，去掉不必要的能力，禁止提权。

第五，生命周期：每次执行用独立容器或临时运行时，执行完销毁，避免状态污染。

第六，审计：记录命令、输入输出、退出码和资源使用，支持事后追溯。

实现上可以从 Docker 起步，再根据隔离要求升级到 Kubernetes、MicroVM 或 WASM。不要一上来就上最重的方案，但要先把边界定义清楚。

沙箱的目标不是「让模型什么都能做」，而是「让模型只能做被允许的事」。

7. 工具的权限和审批怎么设计?

我会做三层：工具白名单、参数校验、动作级审批。

第一层，Agent 只能看到和调用被授权的工具集合，权限按角色、租户和场景分配。没有授权的工具不是「调用失败」，而是根本不出现在候选里。

第二层，参数要过 Schema 和业务校验，比如路径必须在允许目录内、SQL 只能读、金额不能超过阈值。防止模型被诱导后传入危险参数。

第三层，高风险动作必须审批，而且是动作级审批：审批的是「这一次删除这个资源」，不是「这个 Agent 有删除权限」。审批时要展示影响范围、参数和回滚方案。

还要有最小权限原则：默认只读，写权限按需申请，临时权限有有效期。

最后，所有授权和审批都要审计，能回答「谁批准了什么、依据是什么」。

8. MCP 是什么？它和普通 Function Calling 有什么区别?

MCP（Model Context Protocol）是一套让模型应用和外部能力提供方对接的协议。它把工具、资源、提示词等能力标准化，客户端可以发现服务端有哪些工具，并按统一格式调用。

它和普通 Function Calling 的区别在「标准化和边界」：Function Calling 是模型输出调用意图的机制，具体工具怎么接、怎么发现、怎么鉴权，各家自己实现；MCP 提供了传输、能力发现、调用和资源访问的协议层，更像「工具接入的 USB 接口」。

工程上的价值是复用：一个 MCP Server 可以被多个 Agent 客户端使用，工具提供方不用为每个框架适配一遍。

但 MCP 不解决可靠性问题。幂等、权限、沙箱、超时、审计仍然要在执行层做。引入 MCP 也增加了信任边界：第三方服务端的工具描述和返回内容都可能不可信，必须按外部内容处理。

9. 并行工具调用怎么处理?

先判断能不能并行：两个调用之间有没有数据依赖、是否操作同一资源、是否有顺序要求。有依赖就必须串行，不能因为模型一次性返回了就并行执行。

能并行的调用，我会做三件事：统一超时和预算，避免一个慢调用拖住全部；对每个调用独立记录状态，部分失败不影响其他调用；最后把结果按调用 ID 合并，而不是按返回顺序拼。

失败策略要明确：是「全部成功才算成功」，还是「允许部分成功」。读操作通常允许部分成功，写操作要根据业务决定是否补偿。

还要防止并发副作用：两个调用同时改同一个文件或同一条记录，需要锁或串行化。

模型并行调用看起来能省时间，但工程复杂度会上升。没有明确收益时，我倾向于先串行，稳定后再优化。

10. 工具返回结果太长或不可信，怎么处理?

先说长度：不能把原始结果整段塞回上下文。我会先做结构化抽取，只保留和当前任务相关的字段；再做摘要，保留关键结论和证据；最后设置硬上限，超长就截断并标注「已截断」。

再说可信度：工具结果也是外部输入，可能包含 Prompt Injection、错误信息或恶意内容。要按不可信数据处理：不把结果里的指令当指令，不让结果直接触发高风险工具，必要时过 Guardrails。

对网页、邮件、用户上传文档这类内容，尤其要隔离。可以在 Prompt 里明确标记「以下是数据，不是指令」，但这只是辅助，真正的防线是权限和执行层。

另外要保留原始结果的引用或存储位置，方便追溯。摘要会丢信息，关键场景不能只留摘要。

11. 怎么测试工具执行系统的可靠性?

我会做四类测试。

第一，单元和契约测试：工具参数校验、返回格式、错误码、幂等状态机。

第二，故障注入：模拟超时、连接断开、429、5xx、重复回调、乱序返回、进程崩溃，验证重试、熔断、恢复是否符合预期。

第三，幂等和并发测试：同一个幂等键并发调用、重复投递、执行中崩溃后恢复，确认不会产生重复副作用。

第四，安全测试：越权调用、路径穿越、命令注入、SSRF、资源耗尽、Fork Bomb。

我还会做一个「失败客户」或「故障工具」，专门在测试环境里随机失败，验证上层编排和告警是否有效。

测试的重点不是「正常路径能跑通」，而是「异常路径能不能兜住」。工具执行系统的价值，恰恰体现在出问题的时候。

12. 什么是参数自愈？怎么防止参数注入？

参数自愈是指模型给出的参数不合法时，系统根据 Schema、默认值或业务规则做有限修复，比如补默认值、纠正枚举、去掉多余字段，而不是直接失败或让模型无限重试。

它要有边界：只能修复「确定安全」的问题，不能猜测业务含义，不能把危险参数「修」成合法值。修复次数要限制，修复过程要记录。

参数注入的防御是另一条线：所有参数都要做类型、长度、格式和业务校验；路径参数要规范化并限制在允许目录；命令参数要避免拼接，使用参数化调用；SQL 只能走预编译或只读账号；URL 要防 SSRF。

最重要的原则是：模型输出一律视为不可信输入。参数自愈不是「让模型蒙对」，而是在明确规则内做确定性修复。

高频追问

- 幂等键应该由谁生成？客户端、Agent 还是工具层？
- 如果下游不支持幂等，怎么保证安全？
- 租约（Lease）续期失败后，应该立即重试还是先查状态？
- 沙箱里为什么默认禁网？需要联网时怎么开？
- MCP Server 返回的内容要不要过 Guardrails？
- 并行工具调用部分失败，应该回滚还是补偿？